

A2A Protocol: A Developer's Guide

Agent-to-Agent Communication for AI Systems

What is A2A?

A2A (Agent-to-Agent) is an open standard protocol that enables seamless communication and collaboration between AI agents built with different frameworks, by different vendors, and running on separate systems.

Think of A2A as the "HTTP of AI agents" — it provides a common language that allows autonomous AI agents to discover each other's capabilities, delegate tasks, exchange context, and work together on complex user requests — all while preserving their internal opacity (no need to share internal state, memory, or tools).

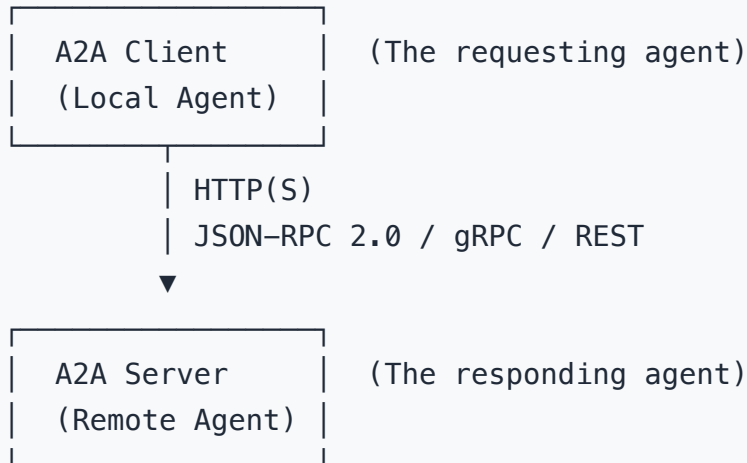
Key Statistics

- **License:** Apache 2.0
 - **Maintained by:** The Linux Foundation (contributed by Google)
 - **Latest Version:** 1.0.0 (March 2026)
 - **SDKs:** Python, Go, JavaScript, Java, .NET
 - **Stars:** 23.9k+ on GitHub
-

Core Concepts

1. Architecture

A2A follows a **client-server architecture**:



2. Agent Discovery: The Agent Card

Every A2A-compliant agent publishes an **Agent Card** at a well-known URL (typically `/.well-known/agent-card`).

This JSON document contains:

- Agent identity (name, description)
- Supported capabilities and skills
- Service endpoint URL
- Authentication requirements (security schemes)
- Supported protocol bindings

Example Agent Card (simplified):

```
{
  "name": "WeatherAgent",
  "description": "An agent that provides weather forecasts",
  "protocolVersion": "2024-06-01",
  "capabilities": {
    "sendMessage": true,
    "sendStreamingMessage": true,
    "getTask": true
  },
}
```

```
"endpoints": {  
  "url": "http://localhost:10001"  
},  
"securitySchemes": [  
  {  
    "type": "http",  
    "scheme": "bearer",  
    "bearerFormat": "JWT"  
  }  
]  
}
```

3. Protocol Bindings

A2A supports multiple transport mechanisms with a **layered architecture**:

Layer 1: Canonical Data Model

- Defined in Protocol Buffers (a2a.proto)
- Protocol-agnostic data structures (Task, Message, Part, Artifact, AgentCard)

Layer 2: Abstract Operations

- `sendMessage` - Send a message and get a response
- `sendStreamingMessage` - Send a message with streaming updates
- `getTask` - Retrieve task state
- `listTasks` - List tasks with filtering
- `cancelTask` - Cancel a running task
- `subscribeToTask` - Subscribe to task updates

Layer 3: Protocol Bindings

1. **JSON-RPC 2.0 over HTTP(S)** - Primary binding
2. **gRPC** - For high-performance scenarios
3. **HTTP+JSON/REST** - Standard RESTful endpoints

4. Custom bindings – Can be created as needed

Data Flow Example:

```
# Client sends a message (JSON-RPC 2.0 style)
POST /sendMessage
Content-Type: application/json

{
  "jsonrpc": "2.0",
  "method": "sendMessage",
  "params": {
    "message": {
      "role": "user",
      "parts": [{"type": "text", "text": "What's the weather in LA?"}]
    }
  },
  "id": "12345"
}
```

4. Task-Centric Design

A2A uses **Tasks** as the fundamental unit of work:

Task
- id: string
- contextId: string (optional)
- status: TaskStatus
- artifacts: list[Artifact]
- history: list[Message]

Task Status States:

- submitted : Task received, queued
- working : Task is being processed
- input_required : Waiting for user input

- completed : Task finished successfully
- failed : Task encountered an error
- canceled : Task was cancelled
- rejected : Task was rejected

5. Streaming & Asynchronous Operations

A2A natively supports long-running operations through:

1. **Server-Sent Events (SSE)** for streaming updates
2. **Push Notifications** via webhook for asynchronous updates

Streaming Example:

```
HTTP/1.1 200 OK
Content-Type: text/event-stream

data: {"event": "task", "task": {...}}
data: {"event": "status", "status": "working", "message": "..."}
data: {"event": "artifact", "artifact": {...}}
data: {"event": "status", "status": "completed"}
```

Building a Multi-Agent System with A2A

Let's explore a practical example: an **Airbnb Planner Multi-Agent System**.

System Overview

This system demonstrates three agents working together:

1. **Host Agent** (Orchestrator)
 - Runs on Google ADK
 - Routes user requests to appropriate specialized agents

- Uses LangGraph for multi-agent orchestration

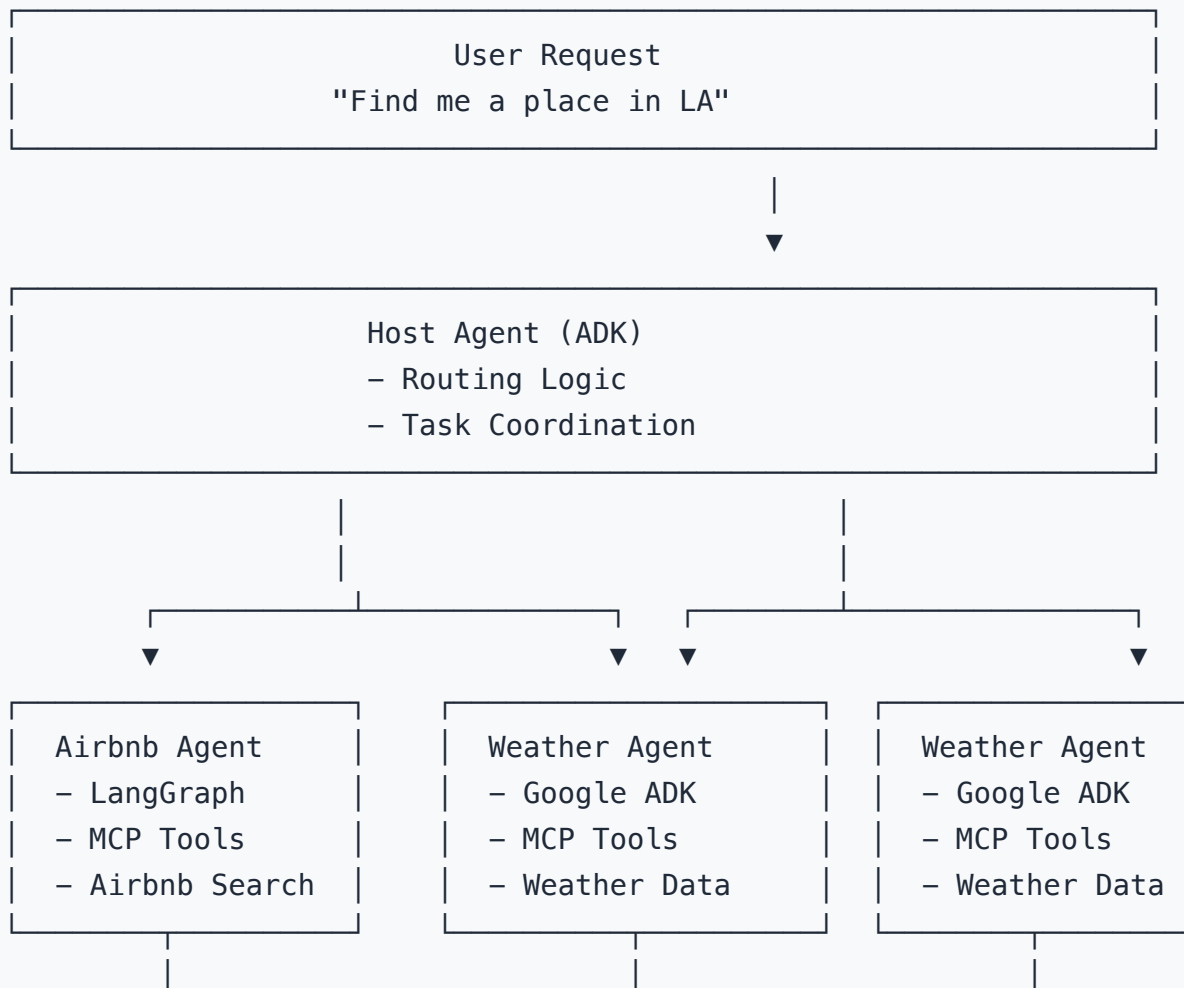
2. Airbnb Agent (Specialized)

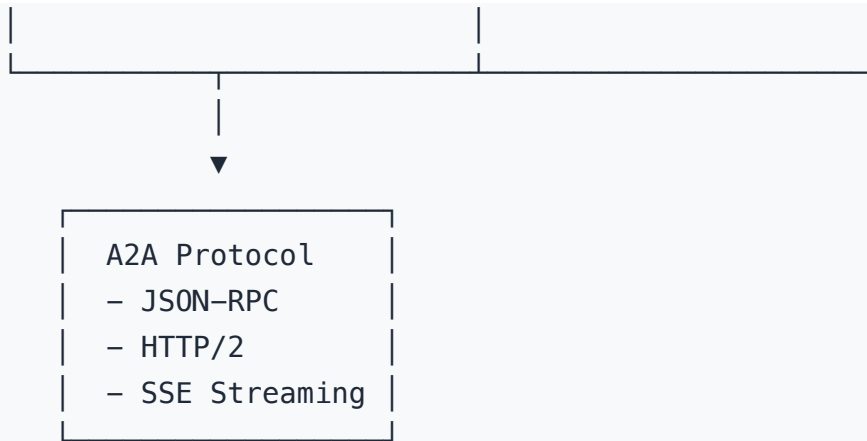
- Uses LangGraph with Google Generative AI
- Integrates with MCP tools for Airbnb search
- Handles accommodation queries

3. Weather Agent (Specialized)

- Uses Google ADK with LiteLLM
- Integrates with MCP tools for weather data
- Handles weather forecast queries

Architecture Diagram





Code Breakdown

1. Host Agent: Agent Card Discovery

The host agent discovers and connects to remote agents:

```
from a2a.client import A2ACardResolver
from a2a.types import AgentCard

async def discover_agents():
    """Discover and connect to remote A2A agents."""
    agents = [
        'http://localhost:10001', # Weather Agent
        'http://localhost:10002', # Airbnb Agent
    ]

    remote_connections = {}
    cards = {}

    async with httpx.AsyncClient(timeout=30) as client:
        for agent_url in agents:
            card_resolver = A2ACardResolver(client, agent_url)

            # Fetch the Agent Card
            card: AgentCard = await card_resolver.get_agent_card()

            # Store connection info
```

```
        remote_connections[card.name] = RemoteAgentConnections(
            agent_card=card,
            agent_url=agent_url
        )
        cards[card.name] = card

    return remote_connections, cards
```

2. Host Agent: Task Routing

The host routes requests to appropriate agents:

```
from google.adk import Agent
from google.adk.tools import tool

class RoutingAgent:
    """Routes user requests to specialized A2A agents."""

    def __init__(self, remote_connections: dict):
        self.connections = remote_connections

    @tool
    async def send_message(self, agent_name: str, task: str):
        """Send a task to a remote A2A agent."""
        client = self.connections[agent_name]

        # Create A2A message request
        payload = {
            'message': {
                'role': 'user',
                'parts': [{'type': 'text', 'text': task}],
                'messageId': str(uuid.uuid4()),
            },
        }

        message_request = SendMessageRequest(
            id=str(uuid.uuid4()),
            params=MessageSendParams.model_validate(payload)
```



```

    )

    # Send via A2A
    response = await client.send_message(message_request)

    return response.root.result.model_dump()

```

3. Airbnb Agent: A2A Server

The Airbnb agent exposes itself as an A2A server:

```

from a2a.server.agent_execution import AgentExecutor, RequestContext
from a2a.server.events.event_queue import EventQueue
from airbnb_agent import AirbnbAgent # Your LangGraph agent

class AirbnbAgentExecutor(AgentExecutor):
    """Executes tasks using the Airbnb LangGraph agent."""

    def __init__(self, mcp_tools: list):
        self.agent = AirbnbAgent(mcp_tools=mcp_tools)

    async def execute(self, context: RequestContext, event_queue: EventQueue):
        """Execute an A2A task."""
        query = context.get_user_input()

        # Stream results from LangGraph agent
        async for event in self.agent.stream(query, context.context_id):
            if event['is_task_complete']:
                # Send final artifact
                await event_queue.enqueue_event(
                    TaskArtifactUpdateEvent(
                        artifact=new_text_artifact(
                            name='result',
                            text=event['content']
                        ),
                        status=TaskState.completed,
                    )
                )

```

```
elif event['require_user_input']:
    # Request user input
    await event_queue.enqueue_event(
        TaskStatusUpdateEvent(
            status=TaskState.input_required,
            message=event['content']
        )
    )
else:
    # Send streaming updates
    await event_queue.enqueue_event(
        TaskStatusUpdateEvent(
            status=TaskState.working,
            message=event['content']
        )
    )
```

4. Weather Agent: ADK Integration

The Weather agent uses Google ADK with MCP tools:

```
from google.adk import Agent
from google.adk.tools.mcp_tool import MCPToolset
from google.adk.models.lite_llm import LiteLlm

def create_weather_agent() -> Agent:
    """Create a weather agent with MCP tools."""
    return Agent(
        model=LiteLlm(model='gemini-2.5-flash'),
        name='weather_agent',
        description='Provides weather forecasts',
        instruction='You are a weather assistant...',
        tools=[
            MCPToolset(
                connection_params=StdioServerParameters(
                    command='python',
                    args=['./weather_mcp.py'], # Your MCP server
                ),
```

```
    )  
  ],  
)
```

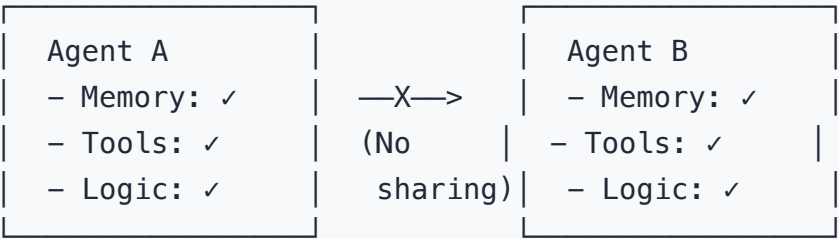
Running the Sample

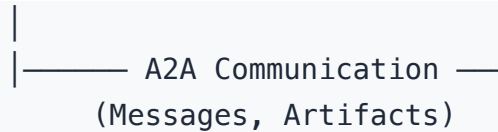
```
# Terminal 1: Start Weather Agent  
cd weather_agent  
uv run .  
  
# Terminal 2: Start Airbnb Agent  
cd airbnb_agent  
uv run .  
  
# Terminal 3: Start Host Agent  
cd host_agent  
uv run .  
  
# Open browser to http://localhost:8083
```

Key Features

1. Opacity Preservation

Agents collaborate **without exposing internal state**:





2. Security

- HTTPS for transport
- JWT authentication via OAuth2/OpenID Connect
- Agent card signing for verification
- Input validation for external data

3. Multi-Turn Support

```
# First message
message_request = SendMessageRequest(
    params=MessageSendParams(
        message={
            'role': 'user',
            'parts': [{'type': 'text', 'text': 'I want to book...'}]
        }
    )
)
response = await client.send_message(message_request)

# Continue conversation with context_id and task_id
message_request = SendMessageRequest(
    params=MessageSendParams(
        message={
            'role': 'user',
            'parts': [{'type': 'text', 'text': 'Actually, change dates.'}],
            'contextId': response.context_id,
            'taskId': response.task_id
        }
    )
)
```

4. Artifact Exchange

Agents can exchange rich content:

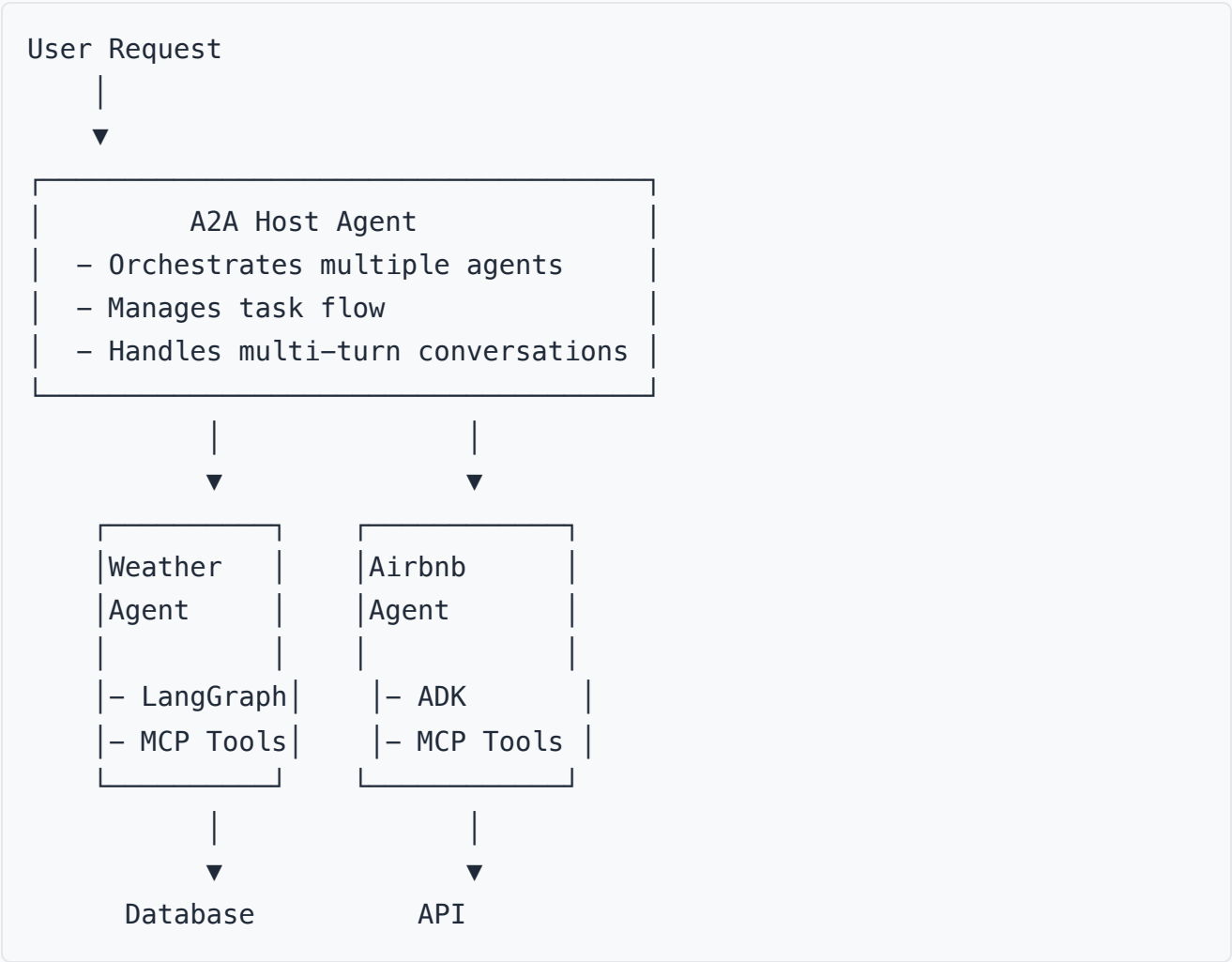
```
{
  "artifacts": [
    {
      "id": "listing_123",
      "type": "text",
      "name": "airbnb_listing",
      "description": "Property details",
      "text": "Spacious 2BR in downtown LA..."
    },
    {
      "id": "map_123",
      "type": "file",
      "name": "location_map",
      "file": {
        "uri": "https://example.com/map.png",
        "mimeType": "image/png"
      }
    }
  ]
}
```

A2A vs MCP: Understanding the Relationship

Aspect	MCP (Model Context Protocol)	A2A (Agent-to-Agent)
Purpose	Connects models to tools & data	Connects agents to each other

Aspect	MCP (Model Context Protocol)	A2A (Agent-to-Agent)
Focus	Model-to-tool communication	Agent-to-agent collaboration
Scope	Single model ↔ multiple tools	Multiple autonomous agents
Complexity	Stateless tool calls	Stateful task management
Use Case	"Query database for user"	"Plan trip: book flights, hotels, tours"

How They Work Together:



Best Practices

1. Agent Design

- **Single Responsibility:** Each agent should have a focused capability
- **Clear Contract:** Document capabilities in Agent Card
- **Error Handling:** Handle failures gracefully with clear messages
- **Idempotency:** Design operations to be safe for retries

2. Security

- **Validate Input:** Always sanitize data from external agents
- **Use HTTPS:** Never send sensitive data over HTTP
- **Rate Limiting:** Implement rate limits to prevent abuse
- **Authentication:** Require auth for production agents

3. Performance

- **Streaming:** Use streaming for long-running operations
- **Caching:** Cache Agent Cards to reduce discovery overhead
- **Async:** Use async I/O for better concurrency
- **Connection Pooling:** Share HTTP clients across connections

4. Monitoring

- **Task Tracking:** Log task lifecycle events
 - **Metrics:** Track response times and success rates
 - **Tracing:** Use distributed tracing for debugging
 - **Alerting:** Alert on task failures or timeouts
-

Getting Started

Install the Python SDK

```
pip install a2a-sdk
```

Create a Simple Agent

```
from a2a.server import A2AServer
from a2a.types import AgentCard, Task

async def main():
    # Create your agent
    agent = YourCustomAgent()

    # Create an A2A server
    server = A2AServer(
        agent=agent,
        card=AgentCard(
            name='MyAgent',
            description='A helpful AI assistant',
            protocolVersion='2024-06-01'
        )
    )

    # Start the server
    await server.serve(host='0.0.0.0', port=8000)

if __name__ == '__main__':
    import asyncio
    asyncio.run(main())
```

Resources

- **Documentation:** <https://a2a-protocol.org>
- **GitHub:** <https://github.com/a2aproject/A2A>
- **Python SDK:** <https://github.com/a2aproject/a2a-python>
- **Samples:** <https://github.com/a2aproject/a2a-samples>
- **DeepLearning.AI Course:** <https://goo.gle/dlai-a2a>

Conclusion

A2A represents a significant step forward in AI agent ecosystems. By providing a standardized protocol for agent-to-agent communication, it enables:

✅ **Interoperability** - Agents built with different frameworks can work together ✅
Scalability - New agents can be added without rewriting existing code ✅ **Security** -
Opacity preservation protects intellectual property ✅ **Flexibility** - Multiple protocol
bindings suit different use cases

Whether you're building a personal assistant, enterprise automation system, or complex multi-agent application, A2A provides the foundation for robust, maintainable agent communication.

A2A JSON Encoding: A Critical Deep Dive

This section clarifies a common misconception about A2A's JSON serialization and addresses the question of encoding safety.

The Short Answer: gRPC is safe, HTTP+JSON uses ProtoJSON (SDK handles it)

The key insight: **A2A does NOT use raw JSON serialization.** Instead, it uses **ProtoJSON** - a specific serialization format defined by Protocol Buffers.

What is ProtoJSON?

ProtoJSON is a standardized JSON encoding for Protocol Buffers. It has specific rules that differ from standard JSON:

Feature	Standard JSON	ProtoJSON
Field names	<code>snake_case</code> → <code>snake_case</code>	<code>snake_case</code> → <code>camelCase</code>
Enum values	UPPERCASE	SCREAMING_SNAKE_CASE
<code>bytes</code> fields	raw bytes or base64	always base64 encoded
<code>google.protobuf.Value</code>	JSON value	JSON-ified value

Source: [ADR-001: Leverage ProtoJSON Specification](#)

How Text Flows Through A2A

gRPC Binding (Binary-Safe)

Model → Raw string → Proto message → gRPC binary → Proto deserialization →

✅ **No escaping concerns** - binary transport handles everything

HTTP+JSON/JSON-RPC Binding (SDK handles ProtoJSON)

Model → Raw string → Pydantic → Proto message → ProtoJSON → JSON → JSON pa

The **SDK** uses `google.protobuf.json_format.MessageToJson()` which:

- 1. Converts Python objects to proto messages
- 2. Applies ProtoJSON rules (`camelCase`, `base64`, `SCREAMING_SNAKE_CASE` enums)
- 3. Returns properly serialized JSON

The Double-Escaping Risk: When It Actually Happens

The risk exists **ONLY** if you bypass the SDK's proto models:

❌ DANGEROUS - Manual JSON Construction

```
# DANGEROUS - Pre-serializing creates double-encoding!
from a2a.types import Part, TextPart

# If LLM returns JSON string...
llm_response = '{"text": "Hello\\nWorld"}' # Already JSON
part = Part(root=TextPart(text=llm_response)) # SDK will JSON-encode A

# Result: "Hello\\\\"nWorld" (double-escaped!)
```

✅ SAFE - Use Proto Models

```
from a2a.types import Part, TextPart

# Pass raw strings directly
part = Part(root=TextPart(text="Hello\\nWorld")) # SDK handles ProtoJSON
# Result: "Hello\\nWorld" (properly escaped once)
```

How to Stay Safe

1. **Use the SDK's proto models** (`TextPart` , `Message` , `SendMessageRequest` , etc.)
2. **Don't manually construct JSON strings** for message parts
3. **Don't embed pre-serialized JSON** from LLMs in your A2A messages
4. **If LLM returns JSON**, parse it first, then extract raw strings:

```
# SAFE - Parse LLM JSON first
import json
from a2a.types import Part, TextPart
```

```
# LLM returns JSON string
llm_response = llm.generate('...') # '{"result": "Hello\nWorld"}'

# Parse first
parsed = json.loads(llm_response)
text = parsed['result'] # Raw string: "Hello\nWorld"

# Now use in proto model
part = Part(root=TextPart(text=text)) # SDK handles ProtoJSON
```

Protocol Binding Comparison

Binding	Serialization	Escaping Risk
gRPC	Proto binary	✅ None - binary-safe
HTTP+JSON/SDK	ProtoJSON via MessageToJson()	✅ None if using SDK
HTTP+JSON/Manual	Raw JSON	❌ High - you must handle escaping

Summary

- **gRPC:** No concerns - binary transport
- **HTTP+JSON with SDK:** No concerns - SDK uses ProtoJSON correctly
- **HTTP+JSON without SDK:** You must handle ProtoJSON rules manually
- **Double-escaping:** Only happens if you manually construct JSON or embed pre-serialized JSON

Building PDFs from Markdown

The A2A Protocol repository uses a simple make-based pipeline for exporting markdown to PDF:

Tools Used

- **Pandoc** - Converts markdown to HTML, DOCX, and PDF (via LaTeX)
- **md-to-pdf** (Node.js) - Converts markdown to PDF via Chromium/Chrome
- **Make** - Orchestrates the build process

Basic Setup

1. Install Pandoc: `brew install pandoc`
2. Install Node.js: <https://nodejs.org>
3. Run: `make` (builds all formats)

Example Makefile Targets

```
# HTML export
make html

# PDF export (via Chromium)
make pdf

# PDF export (via LaTeX)
make pdf-latex

# Clean output
make clean
```

Configuring PDF Output

PDF styling is controlled by:

1. **CSS file** - `resume-html.css` (or similar)

2. **PDF config** - JavaScript file specifying page size, margins, headers/footers

See `scripts/resume-pdf-config.js` for an example configuration.

This guide was created to help developers understand and implement A2A-based systems. For the latest updates, always refer to the official documentation and GitHub repository.